POSTGRESQL · SQL

Upsert without the error

INSERT … ON CONFLICT and EXCLUDED, with every statement run and every output real.

POSTGRESQL · SQL

A plain INSERT fails

ravi@mail.com is already in customers. Insert him again:

```
INSERT INTO customers
VALUES ('ravi@mail.com', 'Hyderabad', 1);
```

```
ERROR:  duplicate key value violates unique
constraint "customers_pkey"
DETAIL:  Key (email)=(ravi@mail.com) already
exists.
```

In a batch load, that one error stops the whole statement.

POSTGRESQL · SQL

Option 1: skip it

```
INSERT INTO customers
VALUES ('ravi@mail.com', 'Hyderabad', 1)
ON CONFLICT (email) DO NOTHING;
```

```
INSERT 0 0
```

No error. 0 rows inserted. The existing row stays exactly as it was.

Tested on PostgreSQL 16

POSTGRESQL · SQL

Option 2: update it

```
INSERT INTO customers AS c
VALUES ('ravi@mail.com', 'Hyderabad', 1)
ON CONFLICT (email) DO UPDATE
SET city   = EXCLUDED.city,
    orders = c.orders + EXCLUDED.orders;
```

```
INSERT 0 1
```

The count includes rows that were updated.

POSTGRESQL · SQL

EXCLUDED is the new row

c is the row already in the table. **EXCLUDED** is the row you tried to insert. Result:

email	city	orders
ravi@mail.com	Hyderabad	4
anu@mail.com	Chennai	1

Pune became Hyderabad. 3 + 1 orders = 4.

Tested on PostgreSQL 16

POSTGRESQL · SQL

Many rows, one statement

```
INSERT INTO customers AS c VALUES
  ('anu@mail.com',   'Chennai', 1),
  ('meena@mail.com', 'Delhi',   1)
ON CONFLICT (email) DO UPDATE
SET orders = c.orders + EXCLUDED.orders
RETURNING email, orders;
```

email	orders
anu@mail.com	2
meena@mail.com	1

anu was updated, meena was inserted.

POSTGRESQL · SQL

Skip updates that change nothing

```
INSERT INTO customers AS c
VALUES ('anu@mail.com', 'Chennai', 0)
ON CONFLICT (email) DO UPDATE
SET city = EXCLUDED.city
WHERE c.city IS DISTINCT FROM EXCLUDED.city;
```

```
INSERT 0 0
```

Same city, so no update. Send 'Mumbai' and you get INSERT 0 1.

POSTGRESQL · SQL

Trap: same key twice

```
INSERT INTO customers AS c VALUES
  ('ravi@mail.com', 'Pune', 1),
  ('ravi@mail.com', 'Goa',  1)
ON CONFLICT (email) DO UPDATE
SET city = EXCLUDED.city;
```

```
ERROR:  ON CONFLICT DO UPDATE command cannot
affect row a second time
```

Remove duplicate keys from your batch before the upsert.

POSTGRESQL · SQL

Recap

- ✔ DO NOTHING skips rows that conflict
- ✔ DO UPDATE changes the existing row
- ✔ EXCLUDED.col is the value you tried to insert
- ✔ DO UPDATE needs a target, like (email)
- ✔ The target needs a unique index or constraint
- ✔ A WHERE clause skips updates that change nothing

POSTGRESQL · SQL

Save this for your next data load

DO NOTHING, DO UPDATE, EXCLUDED and WHERE. The recap is on slide 9.

Tested on PostgreSQL 16